Java Spring ORM

Dr. Öğr. Üyesi Alper Talha KARADENİZ

ORM Nedir?

- ORM (Object-Relational Mapping - Nesne-İlişkisel Eşleme), bir veritabanındaki tabloları ve bu tablolar arasındaki ilişkileri, uygulama kodundaki nesneler ve nesne ilişkileri olarak görmenizi sağlayan bir tekniktir.

ORM’nin İşlevleri

- **Nesne ve Tablo Eşleşmesi:** ORM, veritabanı satırlarını Java nesnelerine, tabloları ise sınıflara dönüştürür. Bu sayede geliştiriciler, SQL sorgularıyla uğraşmak yerine doğrudan Java nesneleriyle çalışabilirler.
- **Karmaşıklığı Azaltma:** Doğrudan JDBC (Java Database Connectivity) kullanmak yerine bir ORM çerçevesi kullanmak, yazılması gereken kod miktarını azaltır ve işlevselliği daha sezgisel hale getirir.

ORM’nin İşlevleri

- **Hibernate Örneği:** En popüler ve yaygın olarak karşılaşılan ORM çerçevesi Hibernate'dir. Hibernate, JDBC'nin üzerine inşa edilmiştir ve verilerin kalıcılığı (persistence) ile çalışma sürecini basitleştirir.
- **Spring ile Entegrasyon:** Spring Framework, Spring Data Access modülü aracılığıyla ORM çerçeveleriyle (özellikle Hibernate) entegrasyon sağlar. Spring Data JPA gibi projeler, bu ORM araçlarını kullanarak veritabanı işlemlerini minimum kodla gerçekleştirmenize olanak tanır.

JPA & Hibernate

- JPA (Java Persistence API veya modern adıyla Jakarta Persistence API), Java uygulamalarında ilişkisel verilerle çalışmak için kullanılan standart bir belirtimdir (specification). Hibernate ise bu belirtimin en yaygın kullanılan uygulamasıdır (implementation) ve bir Nesne-İlişkisel Eşleme (ORM) çerçevesi olarak veritabanı tablolarını Java nesnelerine dönüştürür.

JPA ve Hibernate Arasındaki İlişki

- **Belirtim ve Uygulama:** JPA, verilerin nasıl kalıcı hale getirileceğine dair kuralları koyan bir arayüz kümesidir; Hibernate ise bu kuralları hayata geçiren motor görevi görür.
- **ORM Mantığı:** Hibernate, veritabanı tablolarını ve aralarındaki ilişkileri doğrudan Java nesneleri olarak görmenizi sağlayarak düşük seviyeli SQL ve JDBC karmaşıklığını ortadan kaldırır.
- **Spring Boot Entegrasyonu:** Spring Boot'un JPA starter bağımlılığı (spring-boot-starter-data-jpa), Hibernate'i varsayılan JPA uygulaması olarak otomatik olarak projeye dahil eder.

Spring Data JPA'nın Rolü

- **Otomatik Repozitoriler:** Geliştiriciler, CrudRepository veya JPA'ya özgü ek yetenekler sunan JpaRepository arayüzlerini genişleterek (extending) herhangi bir kod yazmadan temel veri işlemlerini (ekleme, okuma, güncelleme, silme) gerçekleştirebilirler.
- **Entity Anotasyonları:** Veritabanı tabloları ile eşleşecek sınıflar @Entity ile işaretlenir ve birincil anahtar alanı @Id anotasyonu ile tanımlanır.

Spring Data JPA'nın Rolü

- **İlişki Yönetimi:** Tablolar arasındaki ilişkiler @OneToMany, @ManyToOne veya @ManyToMany gibi anotasyonlar kullanılarak nesne yönelimli bir şekilde kurulur.
- **Esnek Sorgulama:** Metot isimlerinden otomatik sorgu üretme (örneğin findByDeliveryZip) özelliğine sahiptir; daha karmaşık ihtiyaçlar için ise @Query anotasyonu ile özel SQL veya JPQL sorguları yazılabilir.

@Entity, @Id, @GeneratedValue

- **@Entity:** Bir Java sınıfını veritabanı tablosuyla eşleşen bir varlık (entity) olarak işaretler. Bu anotasyon, JPA'ya bu sınıfın veritabanında saklanabilir bir nesne olduğunu bildirir. JPA spesifikasyonuna göre, @Entity ile işaretlenmiş sınıfların parametresiz bir yapıcı metoda (no-arguments constructor) sahip olması zorunludur.
- **@Id:** Bir entity içindeki belirli bir alanın, veritabanı tablosundaki birincil anahtar (primary key) olduğunu belirtir. Bu alan, veritabanındaki her bir satırı benzersiz (unique) şekilde tanımlamak için kullanılır.

@Entity, @Id, @GeneratedValue

- **@GeneratedValue:** Birincil anahtar değerlerinin (ID) veritabanı tarafından otomatik olarak üretilmesi gerektiğini belirtir,. Genellikle @Id anotasyonu ile birlikte kullanılır. Örneğin, strategy = GenerationType.AUTO parametresi kullanılarak ID değerinin üretim stratejisi veritabanına veya JPA sağlayıcısına bırakılabilir.

JpaRepository

- JpaRepository, Spring Data JPA'nın sunduğu ve ilişkisel veritabanı işlemlerini büyük ölçüde basitleştiren en güçlü arayüzlerden (interface) biridir. Geliştiricilerin veri erişim katmanında (repository layer) neredeyse hiç kod yazmadan karmaşık veritabanı işlemlerini gerçekleştirmesini sağlar.

Temel Özellikleri ve Hiyerarşisi

- **CrudRepository:** Temel ekleme, okuma, güncelleme ve silme (CRUD) işlevlerini sağlar.
- **PagingAndSortingRepository:** Verileri sayfalama (paging) ve sıralama (sorting) yeteneklerini ekler.
- **JpaRepository Özellikleri:** Bunlara ek olarak, veritabanı durumunu eşitleme (flushing) ve kayıtları toplu silme (delete in batch) gibi JPA'ya özgü gelişmiş yöntemler sunar.

Nasıl Kullanılır?

- **Entity Sınıfı:** Yönetilecek olan varlık sınıfı (örneğin, Ingredient veya Account).
- **ID Tipi:** Bu varlığın birincil anahtarının (@Id) tipi (örneğin, Long veya String).

Sorgu Özelleştirme Yöntemleri

- **Metod İsimlerinden Sorgu Üretme (Query Derivation):** Spring Data, metod isimlerini analiz ederek otomatik olarak SQL sorguları oluşturabilir. Örneğin, findByDeliveryZip(String zip) ismi, arka planda "deliveryZip" alanına göre filtreleme yapan bir sorguya dönüştürülür.
- **@Query Anotasyonu:** Metod ismiyle ifade edilemeyecek kadar karmaşık sorgular için metodun üzerine @Query eklenerek özel SQL veya JPQL sorguları yazılabilir.

Avantajları

- **Boilerplate Kodu Ortadan Kaldırır:** Geleneksel JDBC ile yazılan uzun ve hata yapmaya müsait veritabanı erişim kodlarını yok eder.
- **Bakımı Kolaydır:** Veritabanı işlemleri sadece metod imzalarıyla tanımlandığı için kodun okunabilirliğini artırır.
- **Bağımsızlık Sağlar:** Veritabanı teknolojisinden (MySQL, Postgres, H2 vb.) bağımsız bir programlama modeli sunar.

H2 vs PostgreSQL

Özellik	H2	PostgreSQL
Ana Senaryo	Geliştirme ve Entegrasyon Testleri	Üretim ve Kalıcı Veri Saklama
Çalışma Modu	Bellek içi (In-memory) veya Gömülü	Harici Sunucu
Kurulum	Gerektirmez, bağımlılık olarak eklenir	Sunucu kurulumu ve yapılandırma gerekir
Veri Kalıcılığı	Uygulama kapandığında genellikle silinir	Kalıcıdır

application.yml / application.properties

- Spring Boot uygulamalarında yapılandırma özelliklerini yönetmek için kullanılan iki temel dosya formatı application.properties ve application.yml'dir. Bu dosyalar, uygulamanın davranışını değiştirmek için kullanılan çeşitli özellikleri barındıran merkezi bir konum sunar.

Temel Farklar ve Formatlar

- **application.properties:** Geleneksel anahtar-değer (key-value) çifti formatını kullanır. Örneğin, sunucu portunu değiştirmek için server.port=9090 yazılır.
- **application.yml (YAML):** Hiyerarşik ve daha okunabilir bir yapı sunar.
- Spring Boot her iki formatla da çalışabilir; seçim tamamen geliştiricinin tercihine bağlıdır. Ancak YAML'ın yapısal düzeni, özellikle karmaşık yapılandırmalarda okunabilirliği artırdığı için sıkça tercih edilir.

Kullanım Amaçları

- **Sunucu Ayarları:** Uygulamanın çalışacağı port (varsayılan 8080) veya SSL sertifikası gibi ayarlar yapılabilir.
- **Veritabanı Bağlantısı:** Veritabanı URL'si, kullanıcı adı, şifresi ve sürücü (driver) bilgileri burada tanımlanır.
- **Günlükleme (Logging):** Uygulama paketlerinin veya kök (root) dizininin günlükleme seviyeleri (INFO, DEBUG, WARN vb.) ayarlanabilir.

Kullanım Amaçları

- **Otomatik Yapılandırmayı Özelleştirme:** Spring Boot'un varsayılan olarak sunduğu özellikleri (örneğin Thymeleaf önbelleğini kapatmak gibi) ince ayarlarla değiştirmeye olanak tanır.
- **Actuator Ayarları:** Uygulamanın sağlık durumunu izleyen uç noktaların (endpoints) hangilerinin dışarıya açılacağı buradan yönetilir.

Spring Ortam Soyutlaması (Environment Abstraction)

- Bu dosyalar, Spring'in Environment soyutlamasının bir parçasıdır. Spring; JVM sistem özellikleri, işletim sistemi ortam değişkenleri, komut satırı argümanları ve bu yapılandırma dosyalarındaki tüm değerleri tek bir kaynaktan yöneterek bean'lerin içine enjekte edilmesini sağlar.

Profillerle (Profiles) Kullanım

- **Dosya Bazlı:** application-prod.yml veya application-dev.properties gibi özel isimlendirmelerle profil bazlı ayarlar yapılabilir.
- **YAML ile Tek Dosya:** Sadece YAML formatında geçerli olmak üzere, üç tire (---) kullanılarak farklı profiller tek bir dosya içinde tanımlanabilir.